

CS Fundamentals Note

Binary Operations, Memory Architecture, Solution Design & Time Complexity

1. Binary & Working in Binary

Computers represent, store, and process all instructions using base-2 binary numbers (0s and 1s).

- **Bit & Byte:** A *bit* is the smallest unit of data (0 or 1). A *byte* consists of 8 bits ($2^8 = 256$ possible states).
- **Positional Notation:** Binary uses powers of two ($2^0, 2^1, 2^2, 2^3, \dots$). For example, $1011_2 = (1 \times 8) + (0 \times 4) + (1 \times 2) + (1 \times 1) = 11_{10}$.
- **Bitwise Logic:** Operations like AND, OR, XOR, NOT, and Bit Shifts (`<<`, `>>`) enable direct hardware-level manipulation.

2. Memory Architecture

Understanding how computer memory is organized and accessed is critical for writing efficient software.

Memory Region	Primary Purpose	Characteristics
Stack Memory	Stores local variables and active function call frames.	Fast access, LIFO order, automatic management, fixed size limits.
Heap Memory	Used for dynamic memory allocation during execution.	Flexible size, requires manual allocation/deallocation or garbage collection.
Pointers & Addresses	Holds numeric memory location references.	Enables direct byte indexing and memory manipulation.

3. Defining Solutions & Algorithm Design

Algorithmic problem-solving converts business or computational requirements into structured code.

- **Problem Decomposition:** Breaking down large problems into distinct, modular functions or subroutines.
- **Pseudocode & Flowcharts:** Designing algorithm logic independently of language syntax.
- **Boundary Conditions & Edge Cases:** Ensuring algorithms handle empty lists, null inputs, maximum integer limits, and unexpected inputs gracefully.

4. Time Complexity & Big O Notation

Big O notation models how execution time scales as input size (n) increases.

Big O Notation	Classification	Common Example
O(1)	Constant Time	Accessing an array element by index.
O(log n)	Logarithmic Time	Binary search on a sorted array.
O(n)	Linear Time	Traversing a list with a single loop.
O(n log n)	Linearithmic Time	Efficient sorting algorithms (Merge Sort, Quick Sort).
O(n²)	Quadratic Time	Nested loops over a collection (Bubble Sort).